Unit 4
SMD Practical + Oral Exam Notes

UNIT 4 — OBJECT ORIENTED DESIGN PROCESS

Topic 1: Multi-Layer Architecture (View, Business, Access)

1. Simple Exam Definition:
Multi-layer design splits the system into distinct layers — each with a single responsibility — to improve maintainability, testability, and scalability.

2. Core Concept Explanation:

The 3 Layers:

1. **View Layer (UI/Presentation):** What the user sees and interacts with. Only displays data and captures input. No business logic.
2. **Business Layer (Logic/Domain):** Contains the rules, calculations, and decision-making. Knows nothing about the database or the UI.
3. **Access Layer (Data/Persistence):** Handles all database communication — CRUD operations, SQL queries, ORM mapping.

Why Separation Matters:

- If the UI changes (e.g., switch from web to mobile), only the View Layer changes.
- If business rules change, only the Business Layer changes.
- If the database changes (MySQL → PostgreSQL), only the Access Layer changes.

3. Real Practical Example (Student Management System):

Layer	Class / File	Responsibility
View	`EnrollmentForm.html`, `GradesPage.vue`	Show form, capture clicks
Business	`EnrollmentService.java`, `GradeCalculator.java`	Validate prerequisites, calculate GPA
Access	`EnrollmentDAO.java`, `StudentRepository.java`	INSERT/SELECT from MySQL

4. Examiner Viva Questions:

- What are the 3 layers? What is the role of each?
- Why should SQL NOT be in the View layer?
- What is a DAO class?

5. Strong Viva Answers:

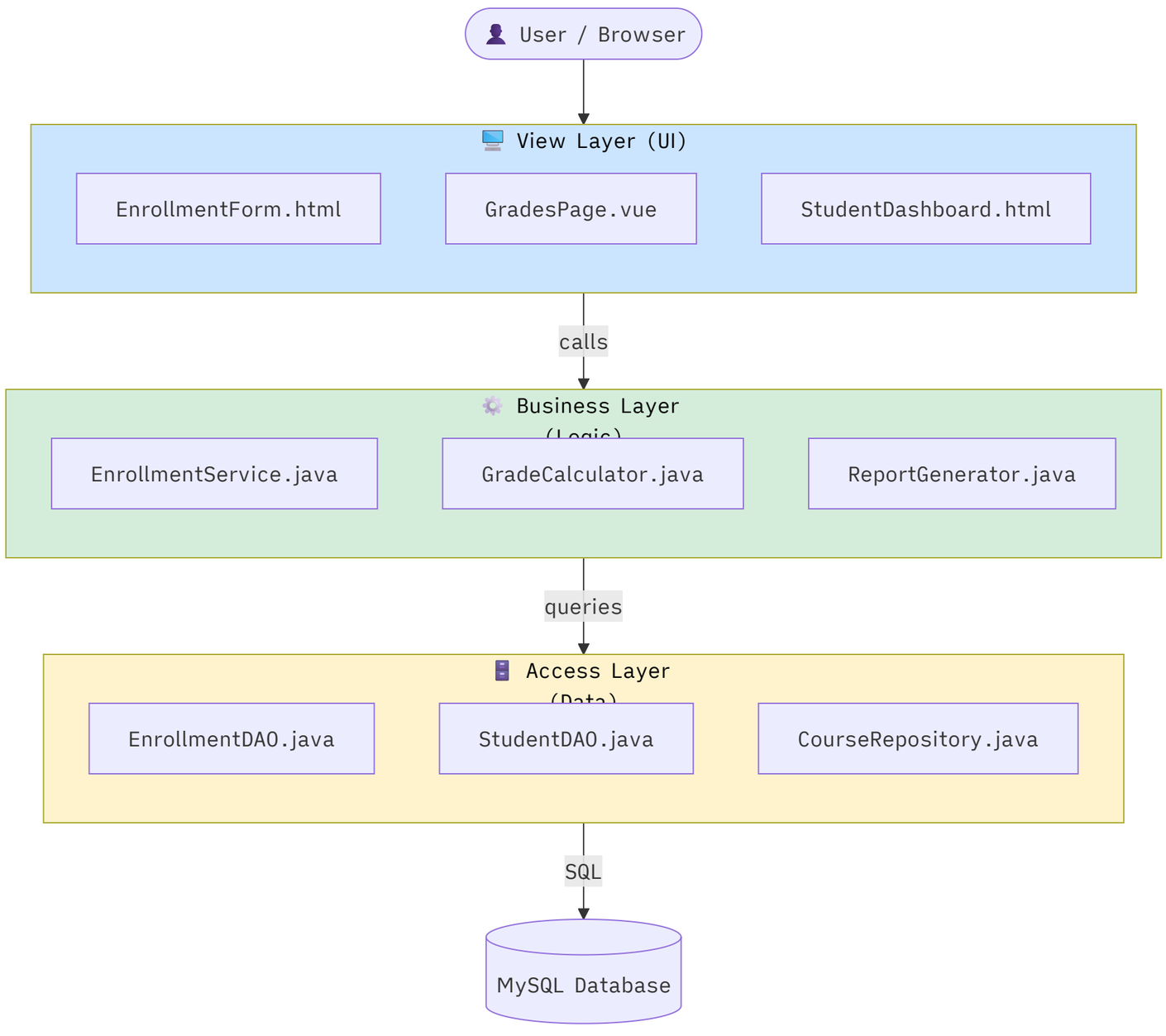
- *Answer:* The three layers are View (UI), Business (Logic), and Access (Data). They separate concerns so each layer can change independently.

- *Answer:* DAO (Data Access Object) is a class in the Access Layer that contains all database queries for a specific entity. Example: `StudentDAO` has `findById()`, `save()`, `delete()` methods.

6. Common Mistakes Students Make:

- Writing SQL queries in UI controllers or View files.
- Putting business logic (e.g., grade calculation) in the DAO.

> 📊 **Visual: 3-Layer Architecture — Student Management System**



Topic 2: Class Visibility & Attribute Refinement

1. Simple Exam Definition:
Class visibility defines who can access attributes and methods of a class. Attribute refinement is the process of adding data types, defaults, and constraints to attributes during the design phase.

2. Core Concept Explanation:

UML Visibility Modifiers (MUST know for viva!):

Symbol	Name	Accessible By
`+`	Public	Everyone (any class)
`-`	Private	Only the class itself
`#`	Protected	Class + Subclasses
`~`	Package / Default	Classes in the same package

Attribute Refinement:
During OOA, attributes are vague (e.g., `name`). During OOD, they are refined:

- `name: String` (add type)
- `age: int = 18` (add default)
- `{age > 0}` (add constraint)
- `{readonly}` (add property)

3. Real Practical Example (Student Class Refinement):

- OOA: `name`, `id`, `grade`
- OOD: `+name: String`, `-password: String = "temp123"`, `+gpa: float = 0.0 {gpa ⩾ 0.0 && gpa ⩽ 10.0}`, `#age: int`

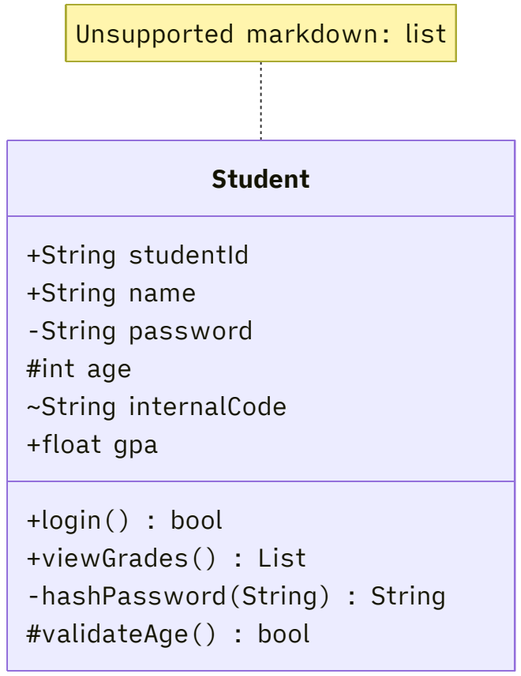
4. Examiner Viva Questions:

- What does `#` mean in UML class notation?
- What is the difference between public and protected?

5. Strong Viva Answers:

- *Answer:* `#` means protected — the attribute is accessible only within the class and its subclasses. A `Student` subclass can access `#age` from `User`, but `Course` class cannot.
- *Answer:* Public (`+`) is accessible by any class. Protected (`#`) is accessible only by the class and its subclasses.

> 📊 **Visual: Class Visibility**



Topic 3: OCL — Object Constraint Language

1. Simple Exam Definition:
OCL (Object Constraint Language) is a formal language used in UML to express constraints and rules on model elements that cannot be expressed using diagrams alone.

2. Core Concept Explanation:
OCL is a declarative language (not executable code). It is used to:

- Add **preconditions** and **postconditions** to operations.
- Add **invariants** (rules that must always be true).
- Navigate associations to express business rules.

OCL Constraint Types:

Type	Description	Example
Invariant	Always true for all instances	`age > 0` on Student
Precondition	Must be true BEFORE operation	`pre: amount > 0`
Postcondition	Must be true AFTER operation	`post: balance = balance@pre - amount`

3. Real Practical Example (Student Management System):

```
-- Invariant: Student GPA must be between 0 and 10
context Student
inv: self.gpa ⩾ 0.0 and self.gpa ⩽ 10.0

-- Precondition: Cannot enroll if no seats available
context Course::enroll(s: Student)
pre: self.availableSeats > 0

-- Postcondition: After enrollment, seat count decreases
context Course::enroll(s: Student)
post: self.availableSeats = self.availableSeats@pre - 1
```

4. Examiner Viva Questions:

- What is OCL? What is it used for?
- What is the difference between an invariant and a precondition?
- Write an OCL constraint for Student GPA.

5. Strong Viva Answers:

- *Answer:* OCL (Object Constraint Language) is a formal text language for specifying constraints on UML models. It is used when a diagram cannot express a business rule precisely.
- *Answer:* An **invariant** must always be true. A **precondition** must be true before an operation runs. A **postcondition** must be true after an operation completes.

6. Common Mistakes Students Make:

- Confusing OCL with a programming language. (OCL does not execute code — it only specifies constraints.)
- Using OCL syntax incorrectly: `self` keyword is mandatory to refer to the current object.

10. Memory Tricks:

- **I**nvariant = **I**s always true.
- **Pre** = Before the method runs.
- **Post** = After the method runs.

Topic 4: ORM — Object-Relational Mapping & Access Layer Design

1. Simple Exam Definition:
ORM (Object-Relational Mapping) is the technique of mapping OO classes to relational database tables, and vice versa, so you can interact with the database using objects rather than SQL directly.

2. Core Concept Explanation:

The Mapping Rules:

OO Concept	Database Equivalent
Class	Table
Object (instance)	Row/Record
Attribute	Column
Association (1-to-many)	Foreign Key
Association (many-to-many)	Junction/Bridge Table
Generalization (inheritance)	Separate tables or merged table

3. Table-Class Mapping (Regular Class):

- `Student` class → `STUDENT` table.
- `Student.studentId` → `STUDENT.student_id` (primary key).
- `Student.name` → `STUDENT.name`.

4. Inherited Class Mapping (Generalization):
Two strategies:

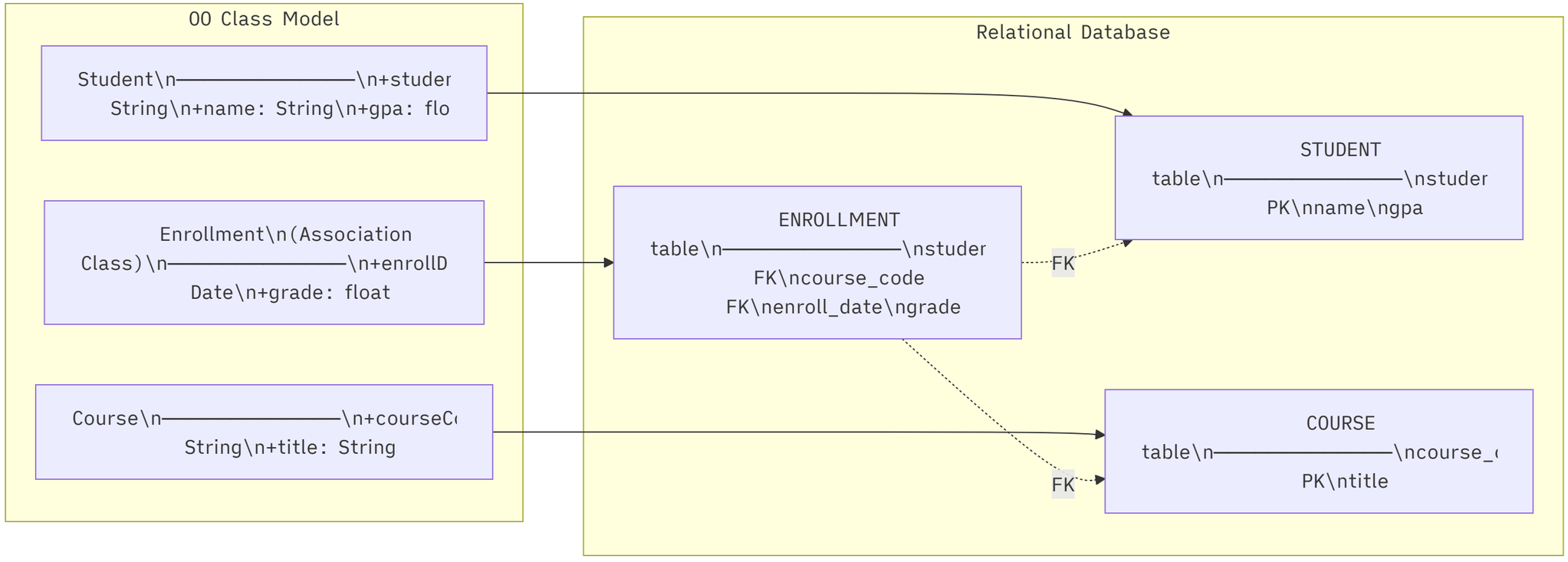
1. **Single Table Strategy:** One table for all classes in hierarchy with a `type` discriminator column. Simple but wastes columns.
2. **Table per Class:** Separate table for each class. Joins needed. More normalized.

Example:

- `User` → `USER(user_id, username, password, type)`.
- `Student` → `STUDENT(user_id FK, student_id, year)`.
- `Professor` → `PROFESSOR(user_id FK, emp_id, department)`.

5. Mirror Classes:
Mirror classes are design classes in the Access Layer that "mirror" (replicate) the structure of business classes, but their purpose is purely data access.
Example: `StudentDAO` mirrors `Student` — it has methods like `findById(id)`, `save(student)`, `delete(id)`.

> 📊 **Visual: ORM — Class to Table Mapping**



6. Examiner Viva Questions:

- What is ORM?
- What is a Mirror Class?
- How is a many-to-many association mapped to a database?

5. Strong Viva Answers:

- *Answer:* ORM maps OO classes to database tables, objects to rows, and attributes to columns. It lets developers work with objects instead of writing raw SQL.
- *Answer:* A many-to-many association needs a **junction table** (also called bridge table or association table) with foreign keys from both tables. Example: `STUDENT_COURSE` table with `student_id` and `course_code` as composite primary key.

Topic 5: Deployment Diagram (CRITICAL FOR VIVA)

1. Simple Exam Definition:
A Deployment Diagram shows the **physical architecture** — which hardware Nodes exist, which software Artifacts run on each Node, and how they communicate.

2. Core Concept Explanation:

Element	Shape	Description
Node	3D box	Physical hardware or execution environment
Artifact	Rectangle with `<<artifact>>` or dog-ear	Actual software file (.jar, .war, .exe)

Element	Shape	Description
Communication Path	Line between nodes	Network connection (HTTP, JDBC, TCP/IP)
Deployment Spec	Property values on artifacts	Execution parameters

3. Real Practical Example (Student Management System):

- **Node 1:** `Client PC` → Artifact: `Web Browser`
- **Node 2:** `Web Server (Apache Tomcat)` → Artifact: `StudentApp.war`
- **Node 3:** `Database Server` → Artifact: `MySQL 8.0`
- Communication: `Client PC` ←HTTP/HTTPS→ `Web Server` ←JDBC→ `DB Server`

4. Examiner Viva Questions:

- What is a Node? What shape represents it?
- What is an Artifact?
- Difference between Component Diagram and Deployment Diagram?

5. Strong Viva Answers:

- *Answer:* A Node is a physical hardware device or software execution environment, drawn as a **3D box** (cube) in UML.
- *Answer:* An Artifact is a physical file (`.jar`, `.war`, `.exe`, `.dll`) that is deployed on a Node.
- *Answer:* A Component Diagram shows **software modules and their dependencies**. A Deployment Diagram shows **physical hardware** and which software runs on which device.

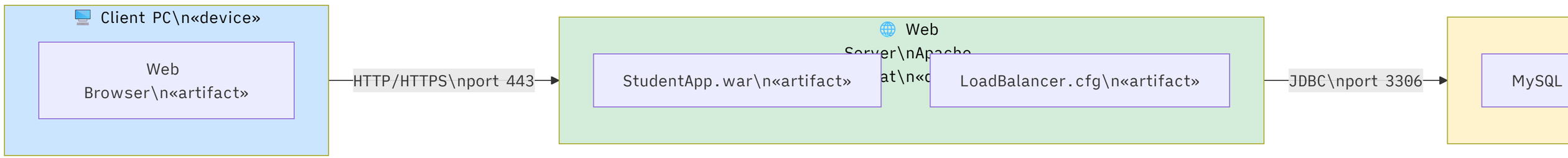
6. Common Mistakes Students Make:

- Drawing flat 2D boxes instead of 3D boxes for Nodes.
- Showing database tables in the Deployment Diagram (only show the DB server Node and DB software Artifact).
- Confusing Component Diagram (software) with Deployment Diagram (hardware).

8. Diagram Drawing Rules:

1. Nodes = 3D cubes (draw a cube shape or labeled box).
2. Artifacts go INSIDE the node box.
3. Label communication paths with protocol names.
4. Use `<<artifact>>` or `<<device>>` stereotypes.

> 📊 **Visual: Deployment Diagram — Student Management System**



9. Difference Tables (Component vs Deployment):

Feature	Component Diagram	Deployment Diagram
Shows	Software modules	Physical hardware
Elements	Components, Interfaces	Nodes, Artifacts
Focus	Software architecture	Physical deployment
Example	UI Component → Business Layer	Client PC → Web Server

Topic 6: Component Diagram

1. Simple Exam Definition:
A Component Diagram shows the **modular software components** of a system and the dependencies between them (which component uses or provides which interface).

2. Core Concept Explanation:

Element	Description
Component	A modular, replaceable software unit (rectangle with component icon)
Interface	What a component provides or requires
Provided Interface	"Lollipop" notation (circle on a stick) — what the component offers
Required Interface	"Socket" notation (half-circle) — what the component needs
Dependency	Dashed arrow between components

3. Real Practical Example (Student Management System):

- `UIComponent` requires `EnrollmentService` interface.
- `BusinessLogic` provides `EnrollmentService` and requires `StudentRepository`.
- `DataAccess` provides `StudentRepository`.

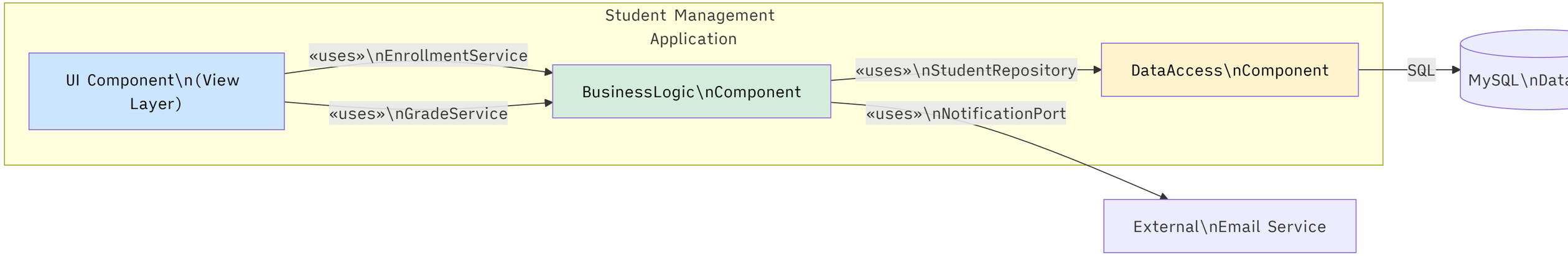
4. Examiner Viva Questions:

- What is the difference between a Class and a Component?
- What is a "provided interface"?

5. Strong Viva Answers:

- *Answer:* A class is a **logical design abstraction**. A component is a **physical, deployable software unit** (like a `.jar` file or a microservice). A component can contain many classes.
- *Answer:* A provided interface (lollipop) is what a component *offers* to others. A required interface (socket) is what a component *needs* from others.

> 📊 **Visual: Component Diagram — Student Management System**



> 💡 **Exam tip:** "Components can be replaced without affecting other components as long as they implement the same interface." This is the key benefit — say this in your viva!